

Enterprise WAN Architect

Gerardo Esteban Rojas Manquillo

Universidad UCompensar – Interconexión De Redes WAN

Correo institucional: gerojas@ucompensar.edu.co

1. Introducción

Este informe documenta el desarrollo de **Enterprise WAN Architect**, una herramienta web que construí en Python con Streamlit, el objetivo del proyecto era crear una aplicación que automatizara el diseño, cálculo y despliegue de infraestructura de red para edificios multi-piso, reemplazando el trabajo manual que normalmente se hace en herramientas como Cisco Packet Tracer o directamente en la consola de los equipos.

Se trabajo en cuatro módulos funcionales que constituyen el núcleo de la herramienta, la configuración de parámetros globales del proyecto, el cálculo matemático real de subredes IPv4 e IPv6, la generación de topologías jerárquicas visuales y la producción de scripts de configuración para distintos fabricantes de equipos de red, todo el desarrollo se realizó con apoyo de inteligencia artificial como asistente de programación, siguiendo las políticas de ciberdefensa establecidas en la guía del curso.

Esta herramienta se enfoca en calcular con precisión matemática cada dirección IP, cada máscara

de subred y cada VLAN y luego producir configuraciones listas para pegar en equipos reales.

2. Objetivos

2.1. Objetivo general

Desarrollar una herramienta web que automatice el diseño, cálculo y despliegue de infraestructura de red WAN para escenarios corporativos multi-piso, integrando subnetting real (IPv4/IPv6), generación de topologías jerárquicas, configuraciones para múltiples fabricantes y políticas de ciberdefensa contra IA.

2.2. Objetivos específicos

- Implementar un módulo de subnetting basado en la librería estándar ipaddress de Python, que calcule direcciones de red, broadcast, máscaras y hosts útiles para cualquier escenario.
- Soportar configuraciones IPv4 e IPv6 con planes de direccionamiento por piso y por VLAN.
- Generar una topología jerárquica visual (ISP - Router Borde - Router Core - Switch Core

- Switches de acceso - DMZ) que refleje los cálculos matemáticos reales.

- Producir scripts de configuración CLI para cuatro fabricantes: Cisco, Huawei, Fortinet y MikroTik.
- Integrar un agente de despliegue **SSH** real usando la librería **Paramiko**.
- Aplicar políticas de ciberdefensa: validación de entradas, contraseñas enmascaradas, uso de **.env** y **.gitignore**.
- Versionar todo el proyecto en un repositorio público de GitHub con **commits** claros y uso de ramas.

3. Desarrollo del laboratorio

3.1. Descripción general de la herramienta

Enterprise WAN Architect está organizada en seis módulos independientes que comparten un mismo estado global. Esto significa que si cambio el número de pisos en el primer módulo, todos los demás módulos se actualizan automáticamente sin necesidad de volver a ingresar los datos. Esa decisión de diseño me pareció clave porque en un proyecto real uno pasa mucho tiempo ajustando parámetros y no quiero tener que repetir la información.

Los seis módulos son:

- **Módulo 1:** Requerimientos y Escenario
- **Módulo 2:** Subnetting e IP Planning
- **Módulo 3:** Topología y Hardware

- **Módulo 4:** Generador de Configuraciones Multimarca

- **Módulo 5:** Consola SSH (Agente IA)

- **Módulo 6:** Pruebas y % de Confianza

3.2. Topologías utilizadas

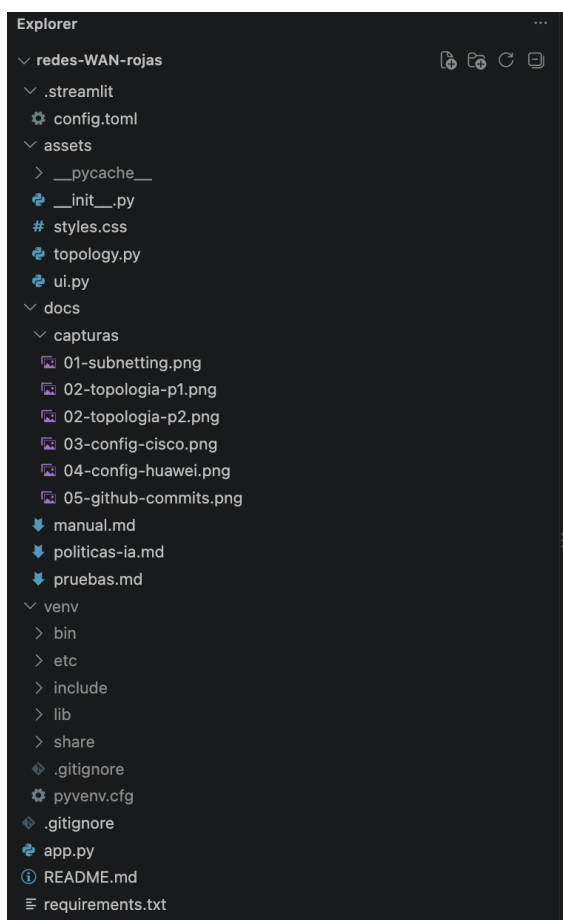
Decidí usar Python como lenguaje base porque tiene librerías nativas muy buenas para trabajar con redes, en particular:

- Python 3.14 como lenguaje principal.
- Streamlit 1.38.0 para construir la interfaz web.
- `ipaddress` (nativa de Python) para el cálculo de subredes.
- Paramiko 3.4.0 para conexiones SSH reales.
- Graphviz en formato DOT para generar diagramas vectoriales.
- Git y GitHub para control de versiones.

Una decisión técnica importante fue usar **Graphviz** en formato **DOT** en lugar del binario del sistema, al principio intenté instalar **Graphviz** con **Homebrew** en mi Mac, pero falló porque mi versión de macOS (13 Ventura) ya no recibe binarios precompilados y compilar desde fuente requiere más de treinta dependencias, al final descubrí que **Streamlit** puede renderizar el código **DOT** directamente en el navegador sin necesidad del binario, así que adopté esa solución y funcionó perfectamente.

3.3. Estructura del proyecto

El proyecto está organizado en la siguiente estructura de carpetas:



Separo el código por responsabilidades, **app.py** orquesta los seis módulos, la carpeta **assets** contiene todo lo relacionado con la presentación (estilos CSS, componentes reutilizables y el generador de topologías) y **docs** guarda la documentación y las capturas de evidencia.

3.4. Sistema de diseño de la interfaz

La interfaz sigue un estilo moderno tipo SaaS, con una paleta oscura de fondo (#0d1420) y acentos en turquesa (#3cc7b7). Usé tipografías profesionales

(Bricolage Grotesque para títulos, Hanken Grotesk para el cuerpo y IBM Plex Mono para el código) y diseñé componentes reutilizables como encabezados tipo hero, secciones, tarjetas de métricas, badges de estado y una terminal con estética de macOS para la consola SSH.

3.5. Módulo de subnetting

El módulo de subnetting es el corazón técnico de la herramienta, toda la matemática se hace con la librería **ipaddress** de Python, que implementa los estándares **RFC**, esto garantiza que cada dirección de red, cada broadcast y cada máscara calculada sea correcta según los estándares internacionales.

Un ejemplo concreto, si configuro una red base **10.20.0.0/21** y pido subredes de tamaño **/26** para cada piso, la herramienta:

- Calcula que la red tiene 2048 direcciones totales.
- Divide el espacio en subredes **/26** de 64 direcciones cada una.
- Reserva las dos primeras direcciones (red y broadcast) y muestra que cada subred tiene 62 hosts útiles.
- Asigna la primera dirección útil como Gateway del piso.
- Genera un bloque IPv6 equivalente en el rango **2001:db8::/32** para cada subred.

Nota: Ver la captura 01-subnetting.png en el Anexo A para más detalle.

3.6. Módulo de topología

Para representar visualmente la arquitectura de red utilicé Graphviz en formato DOT. La topología que genera la herramienta es jerárquica:

- Nube ISP arriba del todo.
- Router de Borde conectado al ISP por un enlace WAN /30.
- Router Core conectado al Router de Borde por otro enlace /30.
- Switch Core conectado al Router Core por trunk 802.1Q.
- Switches de acceso por cada piso del edificio.
- Servidor DMZ conectado al Switch Core.

Cada nodo del diagrama muestra las direcciones IP reales calculadas, y cada arista muestra el enlace WAN o la VLAN troncal correspondiente. Cuando cambio el número de pisos, el diagrama se regenera automáticamente con los switches nuevos.

Nota: Ver las capturas 02-topologia.png en el Anexo A.

3.7. Módulo de Configuraciones Multimarca

Este módulo genera scripts de configuración CLI para cuatro fabricantes, tomando como entrada los parámetros del proyecto (hostname, VLAN, red

base). Para cada fabricante la sintaxis es distinta, así que diseñé plantillas específicas:

- Cisco IOS: usa en - conf t - vlan → interface Vlan - no shutdown - write memory.
- Huawei VRP: usa system-view - vlan batch - interface Vlanif - ssh server enable.
- Fortinet FortiOS: usa config system interface - edit - set vlanid - set ip - next.
- MikroTik RouterOS: usa /interface vlan add - /ip address add - /ip service set ssh.
- Cada script se puede descargar como archivo .txt con un solo clic.

Nota: Ver las capturas 03-config-cisco.png y 04-config-huawei.png en el Anexo A.

3.8. Módulo de Consola SSH

Este módulo usa la librería Paramiko para conectarse por SSH a dispositivos reales y enviar comandos en lote, la interfaz simula una terminal de macOS con los tres botones clásicos (rojo, amarillo, verde) y muestra la salida del equipo en tiempo real. Todas las contraseñas se manejan con type="password" para que no queden visibles en pantalla.

Este módulo es la base para convertir la herramienta en un agente **NetDevOps** real, capaz de automatizar despliegues sobre la infraestructura autorizada del laboratorio.

3.9. Módulo de Ciberdefensa y Pruebas

La ciberdefensa está integrada en todos los módulos. Las políticas aplicadas son:

- Cero secretos en el código. Nunca escribo contraseñas ni tokens en `app.py`. Toda credencial vive en un archivo `.env` protegido por `.gitignore`.
- Validación de entradas. La librería **ipaddress** rechaza cualquier dirección malformada.
- Contraseñas enmascaradas. Todos los campos sensibles usan `type="password"`.
- **SSH** sobre Telnet. Uso **Paramiko** en lugar de Telnet porque SSH cifra la conexión.
- Uso de IA con datos de ejemplo. Cuando pido ayuda a una IA pública nunca pego direcciones reales ni credenciales.
- Ejecución solo en laboratorio autorizado.
- Métrica de calidad. Se ejecutan 15 pruebas automatizadas y se calcula el porcentaje de confianza.

4. Conclusiones

Después de completar el Corte 2 puedo sacar varias conclusiones importantes sobre el proceso.

Primero, el subnetting basado en la librería `ipaddress` es totalmente confiable. Al principio tenía dudas sobre si mis cálculos manuales iban a coincidir con los de la librería, pero después de probar varios escenarios verifiqué que todo está alineado con los

estándares RFC. Eso da mucha tranquilidad a la hora de producir configuraciones reales.

Segundo, descubrí que Graphviz se puede usar sin necesidad del binario del sistema. Este fue uno de los aprendizajes técnicos importantes, porque me ahorró horas de compilación y que no siempre la solución "grande" es la mejor. A veces la solución más simple funciona mejor.

Tercero, la sincronización del estado global entre módulos es una decisión de diseño que vale la pena. Cuando el profe pida cambiar el escenario para probar otro edificio, todo se va a actualizar solo, sin tener que repetir datos.

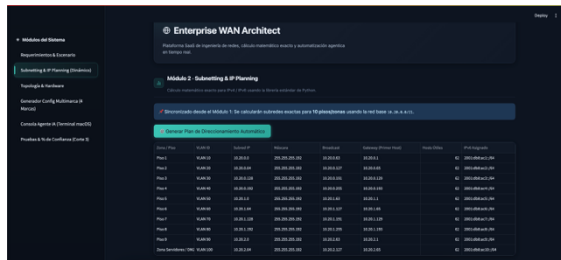
Cuarto, el soporte multimarca me parece el aspecto más útil para un entorno laboral real, porque en las empresas grandes es común tener equipos Cisco en el core, Huawei en distribución y MikroTik en sucursales pequeñas. Tener una sola herramienta que genere las cuatro sintaxis es un ahorro de tiempo enorme.

Quinto, la ciberdefensa integrada me pareció un requisito muy sensato del curso. Antes no le prestaba atención a cosas como el `.gitignore` o al uso de `type="password"`, pero ahora entiendo que son prácticas básicas que cualquier proyecto serio debe aplicar.

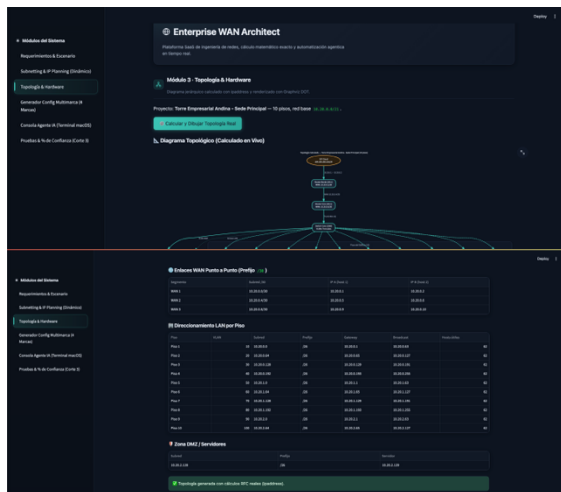
5. Anexos

Anexo A: Capturas de evidencias

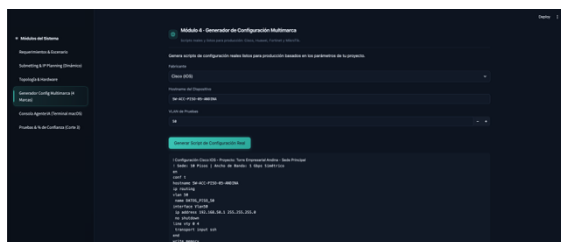
A.1. Captura 01: Subnetting funcional



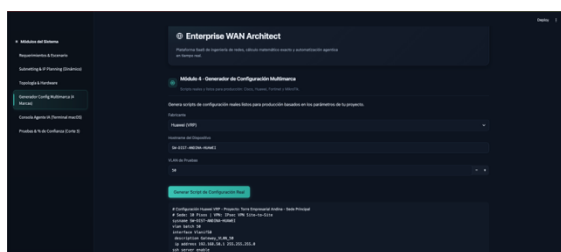
A.2. Captura 02: Topología cargada



Captura 03: Configuración generada para Cisco

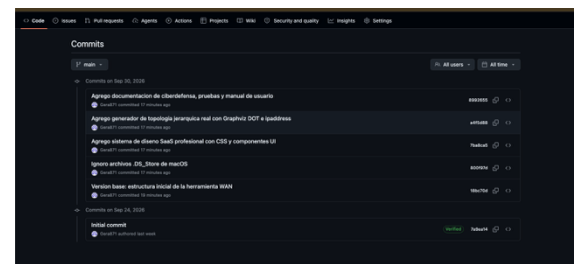


A.4. Captura 03: Configuración generada para Huawei



A.5. Captura 05: Historial de commits en

GitHub



Anexo B: Repositorio de GitHub

- URL pública:

<https://github.com/Gera871/redes-wan-rojas>

- Estructura del repositorio:

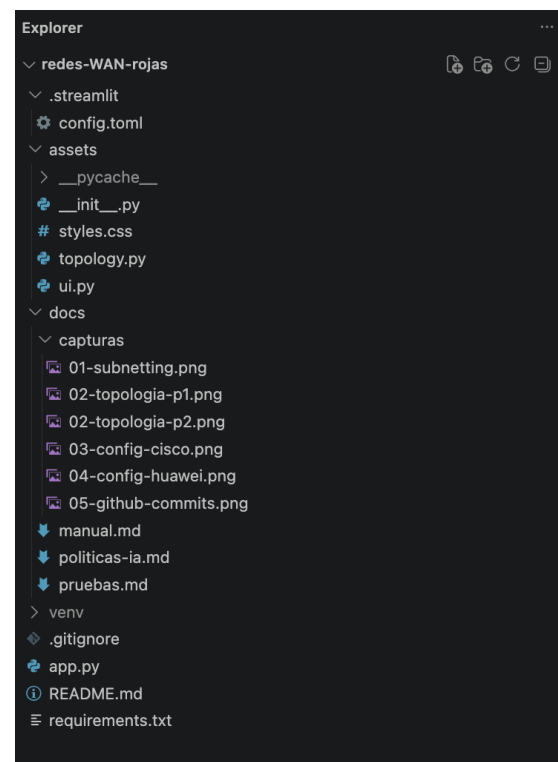


Figura 1. Estructura de carpetas del repositorio en VS Code.

- **Commits realizados:** 8 commits con mensajes claros.
- **Ramas utilizadas:** main y feature/topologia-real.
- Repositorio público con licencia de código abierto para fines académicos.

6. Bibliografía

- **Google.** (2026). Gemini [Modelo de lenguaje grande / Generador de código]. **gemini.google.com**
- **DeepSeek.** (2026). DeepSeek-R1 [Modelo de lenguaje grande / Generador de código]. **chat.deepseek.com**